Observer

Observer · Fan out video.published to audit+UI · Build with Waqar

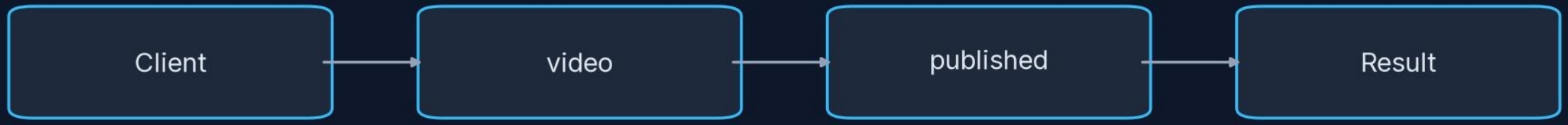
Today's Hook

- Problem: Fan out video.published to audit+UI
- Pattern: Observer — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Fan out video.published to audit+UI
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Observer: Fan out video.published to audit+UI

Real-World Scenario

- Running example: Fan out video.published to audit+UI
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.gof.behavioral.observer.problem;

import java.util.*;

public final class NaivePublish {
  public static void publish(String id){ System.out.println("audit "+id); System.out.println("ui "+id); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.observer.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

interface Listener { void on(String event, String id); }
final class EventBus {
  private final Map<String, List<Listener>> subs = new HashMap<>();
  void on(String event, Listener l){ subs.computeIfAbsent(event,k->new ArrayList<>()).add(l); }
  void publish(String event, String id){
    for (var l: subs.getOrDefault(event, List.of())) l.on(event, id);
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.observer.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Observer: video.published fan-out ===");
    System.out.println("--- naive ---"); NaivePublish.publish("v1");
System.out.println("--- observer ---");
var bus=new EventBus(); List<String> log=new ArrayList<>();
bus.on("video.published", (e,id)->log.add("audit:"+id));
bus.on("video.published", (e,id)->log.add("ui:"+id));
bus.publish("video.published","v1");
bus.publish("other","x");
System.out.println(log);
  }
}
```

Walkthrough Output

```
=== Observer: video.published fan-out ===
--- naive ---
audit v1
ui v1
--- observer ---
[audit:v1, ui:v1]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Observer.

Related Patterns

- Contrast Observer with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Observer → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.